

LAPORAN PRATIKUM STRUKTUR DATA

PEKAN 3

disusun Oleh:

Habib Al Faruq

2511532010

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: Muhammad Galid Avero



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum struktur data mengenai penggunaan struktur data Stack di Java. Penulisan laporan ini bertujuan untuk membuat sebuah laporan tentang materi pemrograman struktur data Stack di java.

Penyusunan laporan praktikum ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada dosen pengampu mata kuliah praktikum pemrograman algoritma dan pemrograman, Bapak Dr. Wahyudi. S.T.M.T dan asisten praktikum, Uda Muhammad Galid Averro yang telah memberikan arahan dan masukan dalam proses penyusunan laporan praktikum ini. Penulis berharap laporan praktikum ini dapat memberikan manfaat bagi pembaca dalam mempelajari konsep penggunaan struktur data Stack di Java.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Praktikum	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	2
BAB II PEMBAHASAN.....	3
2.1 latihanStack_2511532010	3
2.2 NilaiMkasimum_2511532010	5
2.3 SiswaStack_2511532010.....	8
2.4 stackArray_2511532010 dan stackArrayDriver_2511532010.....	12
2.5 StackPostfix_2511532010.....	16
BAB III KESIMPULAN.....	20
3.1 Ringkasan Hasil Praktikum.....	20
DAFTAR PUSTAKA.....	21
TUGAS	22

BAB I

PENDAHULUAN

1.1 Latar Belakang Praktikum

Dalam pengembangan perangkat lunak, struktur data menjadi pilar utama yang menentukan seberapa efisien sebuah program mengelola informasi. Di antara berbagai jenis yang ada, Stack atau tumpukan merupakan struktur data linier yang paling mendasar untuk kita kuasai. Karakteristik utamanya terletak pada terorganisir (Last In First Out), di mana data yang paling akhir ditambahkan justru menjadi yang pertama untuk diakses atau dikeluarkan. Logika ini serupa dengan tumpukan piring atau buku di kehidupan nyata; kita secara alami akan mengambil barang dari posisi paling atas sebelum menjangkau bagian bawahnya.

Relevansi konsep Stack sangat terasa dalam arsitektur komputasi modern. Banyak fitur yang kita gunakan sehari-hari bergantung pada mekanisme ini, mulai dari fungsi undo di aplikasi editor teks, navigasi back pada peramban web, hingga pengaturan alokasi memori saat sistem operasi memproses pemanggilan fungsi. Melalui praktikum ini, kita akan mempelajari bagaimana algoritma bekerja di balik layar dalam mengelola memori serta mengatur urutan eksekusi program secara lebih terorganisir dan sistematis.

1.2 Tujuan Praktikum

Fokus utama praktikum ini adalah memberikan pengalaman praktis dalam mengimplementasikan konsep Stack ke dalam kode program yang fungsional. Kita ditargetkan untuk tidak sekadar memahami teori LIFO secara tekstual, tetapi juga harus menguasai konstruksi operasi-operasi fundamentalnya. Hal ini mencakup kemampuan membangun fungsi push untuk menyisipkan data serta pop untuk mengambil data dari tumpukan. Selain itu, kita akan mempelajari cara memvalidasi kondisi tumpukan

melalui metode `isEmpty` dan `isFull`, sebuah langkah krusial untuk mencegah terjadinya kesalahan logika (logic error) saat aplikasi sedang dieksekusi.

Di sisi lain, praktikum ini dirancang untuk menajamkan kemampuan analitis kita dalam menentukan strategi implementasi yang paling efisien. Kita akan mengevaluasi efektivitas penggunaan array statis yang memiliki kapasitas tetap dibandingkan dengan penggunaan Linked List yang menawarkan fleksibilitas memori secara dinamis. Dengan membedah karakteristik kedua metode tersebut, kita akan membangun fondasi yang kuat dalam memilih solusi pemrograman yang paling optimal saat menghadapi manajemen data kompleks yang membutuhkan pengaturan urutan yang presisi.

1.3 Manfaat Praktikum

Keuntungan utama yang kita peroleh dari praktikum ini adalah penguatan logika pemrograman, khususnya dalam aspek manajemen memori. Kita akan terlatih untuk berpikir secara prosedural dalam memetakan alur masuk dan keluar data pada sebuah sistem. Lebih dari itu, pemahaman mendalam mengenai Stack memberikan wawasan konkret tentang mekanisme internal komputer, seperti runtime stack. Pengetahuan ini sangat krusial bagi kita saat melakukan proses debugging atau mengurai kesalahan pada struktur kode yang kompleks.

Di samping aspek teknis, praktikum ini membekali kita dengan kompetensi standar yang relevan dengan tuntutan industri teknologi. Berbagai algoritma tingkat lanjut, seperti Depth First Search (DFS) pada struktur data Graph hingga evaluasi ekspresi matematika, sangat bergantung pada prinsip dasar Stack. Dengan menguasai konsep ini, kita akan lebih percaya diri dalam menghadapi tantangan pemrograman yang lebih rumit serta memiliki keunggulan kompetitif saat menempuh seleksi teknis dalam jenjang karir profesional sebagai pengembang perangkat lunak.

BAB II

PEMBAHASAN

2.1 latihanStack_2511532010

```
1 package pekan3_2511532010;
2 import java.util.Stack;
3 public class latihanStack_2511532010 {
4
5     public static void main(String[] args) {
6         Stack<Integer> s = new Stack<Integer> ();
7         s.push(42);
8         s.push(-3);
9         s.push(17);
10
11         System.out.println("Nilai statck = " + s);
12         System.out.println("Nilai pop = " + s.pop());
13         System.out.println("Nilai stack setelah pop = " + s);
14         System.out.println ("nilai peek" +s.peek());
15         System.out.println("Nilai stack setelah peek" +s);}}
16
```

1. Deklarasi dan Inisialisasi Stack

Pada baris `Stack<Integer> s = new Stack<Integer> ()`, kita membuat sebuah objek tumpukan baru. Kita menggunakan wrapper class `Integer` karena koleksi dalam Java hanya dapat menyimpan objek, bukan tipe data primitif `int`. Objek `s` inilah yang nantinya akan mengatur alur data masuk dan keluar secara otomatis.

2. Operasi Push (Menambah Data)

Kita memasukkan tiga buah angka secara berurutan: 42, -3, dan 17 menggunakan metode `.push()`. Dalam struktur Stack, angka 42 akan berada di posisi paling bawah, diikuti -3, dan angka 17 akan berada di posisi paling atas (top). Ini adalah implementasi langsung dari cara kerja tumpukan.

3. Representasi String Bawaan

Saat kita menjalankan `System.out.println(s)`, Java secara otomatis memanggil metode `toString()` dari kelas `Stack`. Hasilnya akan ditampilkan dalam kurung siku, misalnya `[42, -3, 17]`. Urutan ini menunjukkan elemen dari yang paling lama dimasukkan hingga yang terbaru di posisi paling kanan.

4. Operasi Pop (Mengambil Data)

Baris `s.pop()` digunakan untuk mengambil elemen yang berada di posisi paling atas. Karena angka 17 adalah yang terakhir kita masukkan, maka angka itulah yang akan dikeluarkan dan dihapus dari tumpukan. Setelah operasi ini, tumpukan kita hanya akan menyisakan `[42, -3]`.

5. Prinsip LIFO dalam Praktik

Melalui urutan eksekusi di atas, kita bisa melihat bukti nyata prinsip LIFO (Last In First Out). Meskipun angka 42 dimasukkan pertama kali, ia tertahan di bawah dan tidak bisa diambil sebelum elemen-elemen di atasnya dikeluarkan. Angka 17 yang datang terakhir justru menjadi yang pertama keluar.

6. Operasi Peek (Mengintip Data)

Metode `s.peek()` berfungsi untuk melihat siapa yang berada di posisi teratas tanpa harus menghapusnya. Dalam kode kita, setelah angka 17 di-pop, posisi teratas sekarang ditempati oleh -3. Saat peek dipanggil, ia akan menampilkan -3 namun tidak mengubah isi tumpukan tersebut.

HASIL :

```
Nilai statck = [42, -3, 17]
Nilai pop = 17
Nilai stack setelah pop = [42, -3]
nilai peek-3
Nilai stack setelah peek[42, -3]
```

2.2 NilaiMaksimum_2511532010

```
1 package pekan3_2511532010;
2 import java.util.Stack;
3 public class NilaiMaksimum_2511532010 {
4     public static int max(Stack<Integer> s ) {
5         Stack<Integer> backup = new Stack<Integer>();
6         int maxValue = s.pop();
7         backup.push(maxValue);
8         while (!s.isEmpty()) {
9             int next = s.pop();
10            backup.push(next);
11            maxValue = Math.max(maxValue, next);
12        }
13        while (!backup.isEmpty()) {
14            s.push(backup.pop());
15        }
16        return maxValue;
17    }
18    public static void main(String[] args) {
19        Stack<Integer> s = new Stack<Integer>();
20        s.push(70);
21        s.push(12);
22        s.push(20);
23        System.out.println ("isi statck " + s);
24        System.out.println ("Stack Teratas " + s.peek());
25        System.out.println ("Nilai maksimum "+ max(s));}}
```

1. Tujuan Utama Metode Max

Kita membuat metode statis max yang dirancang untuk menyisir seluruh isi Stack guna menemukan angka tertinggi. Masalah utama yang kita hadapi pada Stack adalah kita tidak bisa langsung melihat data di tengah atau di bawah; kita harus membongkarnya satu per satu dari atas.

2. Strategi Backup Data

Poin paling krusial di sini adalah penggunaan `Stack<Integer> backup`. Karena kita harus melakukan `pop()` untuk memeriksa setiap angka, data asli di dalam Stack `s` akan hilang. Oleh karena itu, kita menyiapkan tumpukan cadangan agar data yang kita bongkar tidak hilang begitu saja.

3. Inisialisasi Nilai Maksimum

Kita memulai pencarian dengan mengambil elemen teratas pertama menggunakan `s.pop()` dan menyimpannya ke variabel `maxValue`. Secara logika, sebelum angka lain diperiksa, angka pertama ini adalah nilai terbesar sementara yang kita miliki. Angka ini juga langsung kita masukkan ke dalam backup.

4. Proses Iterasi dengan While-Loop

Kita menggunakan perulangan `while (!s.isEmpty())` untuk mengosongkan tumpukan utama. Selama masih ada angka di dalam Stack `s`, program akan terus mengambil data tersebut. Ini adalah cara standar bagi kita untuk melakukan penelusuran (traversal) pada struktur data Stack.

5. Logika Perbandingan (`Math.max`)

Di dalam perulangan, kita menggunakan fungsi `Math.max(maxValue, next)`. Setiap angka yang baru kita keluarkan dari tumpukan akan dibandingkan dengan pemegang rekor `maxValue`. Jika angka baru tersebut lebih besar, maka `maxValue` akan diperbarui secara otomatis.

6. Pengamanan Elemen ke Stack Backup

Setiap kali kita mengeluarkan angka dari Stack utama untuk dibandingkan, kita segera memasukkannya ke `backup.push(next)`. Ini memastikan bahwa meskipun Stack utama kosong setelah proses pencarian, semua data kita tersimpan dengan aman di tumpukan cadangan, namun dalam urutan yang terbalik.

7. Restorasi (Mengembalikan Data)

Setelah mendapatkan nilai maksimum, Stack utama `s` dalam kondisi kosong. Untuk itu, kita melakukan perulangan kedua menggunakan `while (!backup.isEmpty())`. Di sini kita memindahkan kembali semua data dari `backup` ke `s`. Proses ini mengembalikan tumpukan ke posisi dan urutan semula.

8. Efisiensi Penggunaan Memory

Dengan metode ini, kita menggunakan ruang tambahan sebesar $O(n)$ untuk tumpukan cadangan. Meskipun terlihat repot karena harus memindahkan data bolak-balik, ini adalah satu-satunya cara bagi kita untuk memproses data Stack tanpa menghilangkan isinya (bersifat non-destructive).

9. Demonstrasi pada Main Method

Dalam metode `main`, kita memasukkan angka 70, 12, dan 20. Perlu kita ingat bahwa dalam tumpukan, 20 berada di paling atas. Program akan mencetak isi tumpukan dan membuktikan bahwa meskipun angka 70 berada di posisi paling bawah, algoritma kita tetap bisa menemukannya.

10. Validasi Akhir Melalui Peek

Setelah pemanggilan `max(s)`, kita bisa yakin bahwa isi tumpukan tetap utuh. Jika kita melakukan `s.peek()` setelah mendapatkan nilai maksimum, angka yang muncul tetaplah 20. Ini membuktikan bahwa proses backup dan restorasi yang kita rancang berjalan dengan sempurna tanpa mengubah struktur data aslinya.

HASIL :

```
isi statck [70, 12, 20]  
Stack Teratas 20  
Nilai maksimum 70
```

2.3 SiswaStack_2511532010

```

1 package pekan3_2511532010;
2
3 import java.util.ArrayList;
4
5 class Siswa_2511532010 {
6     String nama;
7     int nim;
8
9     public Siswa_2511532010(String nama, int nim) {
10         this.nama = nama;
11         this.nim = nim;
12     }
13
14     @Override
15     public String toString() {
16         return "Nim: " + nim + ", Nama: " + nama;
17     }
18 }
19
20 public class SiswaStack_2511532010 {
21     private ArrayList<Siswa_2511532010> stack;
22
23     public SiswaStack_2511532010() {
24         stack = new ArrayList<>();
25     }
26
27     public void push_2511532010(Siswa_2511532010 mhs) {
28         stack.add(mhs);
29     }
30
31     public Siswa_2511532010 pop_2511532010() {
32         if (!isEmpty_2511532010()) {
33             return stack.remove(stack.size() - 1);
34         }
35         return null;
36     }
37
38     public Siswa_2511532010 peek_2511532010() {
39         if (!isEmpty_2511532010()) {
40             return stack.get(stack.size() - 1);
41         }
42         return null;
43     }
44
45     public boolean isEmpty_2511532010() {
46         return stack.isEmpty();
47     }
48
49     public void tampilkanSiswa_2511532010() {
50         for (int i = stack.size() - 1; i >= 0; i--) {
51             System.out.println(stack.get(i));
52         }
53     }
54
55     public static void main(String[] args) {
56         SiswaStack_2511532010 studentStack = new SiswaStack_2511532010();
57
58         Siswa_2511532010 mhs1 = new Siswa_2511532010("Ali", 1);
59         Siswa_2511532010 mhs2 = new Siswa_2511532010("Boby", 2);
60         Siswa_2511532010 mhs3 = new Siswa_2511532010("Charles", 3);
61
62         studentStack.push_2511532010(mhs1);
63         studentStack.push_2511532010(mhs2);
64         studentStack.push_2511532010(mhs3);
65
66         System.out.println("Siswa di dalam stack");
67         studentStack.tampilkanSiswa_2511532010();
68
69         System.out.println("Siswa teratas " + studentStack.peek_2511532010());
70         System.out.println("Mengeluarkan siswa dari stack : " + studentStack.pop_2511532010());
71
72         System.out.println("Daftar siswa setelah di pop : ");
73         studentStack.tampilkanSiswa_2511532010();
74     }
75 }

```

1. Representasi Objek Siswa

Kita membuat kelas `Siswa_2511532010` sebagai model data dengan atribut nama dan nim. Di sini, kita juga melakukan override metode `toString()` agar saat data siswa dicetak, formatnya terlihat rapi dan informatif, bukan sekadar kode alamat memori.

2. Implementasi Custom Stack

Dalam kelas `SiswaStack_2511532010`, kita menggunakan `ArrayList<Siswa_2511532010>` sebagai media penyimpanan internal. Ini menunjukkan fleksibilitas kita dalam memanfaatkan daftar dinamis untuk berperilaku seperti tumpukan statis dengan aturan akses yang kita batasi sendiri.

3. Logika Push Manual

Metode `push_2511532010` yang kita buat berfungsi untuk menambahkan elemen ke dalam daftar. Karena kita membangun Stack, maka setiap data baru akan ditambahkan ke posisi paling akhir dari `ArrayList`. Posisi terakhir inilah yang secara logis kita anggap sebagai "puncak tumpukan".

4. Operasi Pop dan Manajemen Indeks

Pada metode `pop_2511532010`, kita mengambil data dari indeks terakhir, yaitu `stack.size() - 1`. Setelah diambil, data tersebut langsung dihapus dari daftar. Ini adalah inti dari prinsip LIFO (Last In First Out), di mana elemen yang terakhir masuk adalah yang pertama kali kita buang.

5. Keamanan Data dengan `isEmpty`

Kita sangat memperhatikan aspek keamanan program dengan menyertakan pengecekan `isEmpty_2511532010()`. Sebelum melakukan `pop` atau `peek`, kita memastikan dulu bahwa tumpukan tidak kosong. Jika kita langsung mengambil data dari tumpukan kosong, program akan mengalami error (indeks di luar batas).

6. Fungsi Peek (Mengintip)

Metode `peek_2511532010` yang kita rancang hampir sama dengan `pop`, namun bedanya kita hanya menggunakan `.get()` tanpa menghapus datanya. Ini memungkinkan kita untuk melihat siapa siswa yang berada di posisi teratas tanpa mengubah kondisi tumpukan sama sekali.

7. Iterasi Terbalik pada TampilkanSiswa

Poin yang menarik adalah pada metode `tampilkanSiswa_2511532010`. Kita menggunakan perulangan `for` yang berjalan mundur (dari indeks terbesar ke nol). Hal ini kita lakukan agar urutan tampilan di layar sesuai dengan konsep tumpukan: data yang paling atas (terbaru) muncul paling pertama.

8. Fleksibilitas Tipe Data Objek

Berbeda dengan latihan sebelumnya yang hanya menggunakan Integer, kali ini kita membuktikan bahwa Stack bisa mengelola objek yang kompleks. Ini adalah simulasi nyata bagaimana sistem komputer mengelola data pengguna atau proses antrian dalam sebuah aplikasi.

9. Simulasi pada Main Method

Di dalam `main`, kita memasukkan Ali, Bobby, dan Charles secara berurutan. Berdasarkan logika LIFO, Charles yang masuk terakhir akan berada di posisi paling

atas. Saat kita memanggil peek, Charles-lah yang akan muncul sebagai siswa teratas.

10. Hasil Akhir Operasi Pop

Setelah kita menjalankan pop, Charles dikeluarkan dari daftar. Ketika kita menampilkan kembali isi tumpukan, siswa yang tersisa hanya Bobby dan Ali. Poin ini mempertegas pemahaman kita bahwa dalam Stack, elemen hanya bisa keluar dari satu pintu, yaitu pintu yang sama tempat elemen tersebut masuk.

HASIL :

```
Siswa di dalam stack
Nim: 3, Nama: Charles
Nim: 2, Nama: Bobby
Nim: 1, Nama: Ali
Siswa teratas Nim: 3, Nama: Charles
Mengeluarkan siswa dari stack : Nim: 3, Nama: Charles
Daftar siswa setelah di pop :
Nim: 2, Nama: Bobby
Nim: 1, Nama: Ali
```

2.4 stackArray_2511532010 dan stackArrayDriver_2511532010

```

1 package pekan3_2511532010;
2
3 public class stackArray_2511532010 {
4     static final int MAX = 1000;
5     int top;
6     int a[] = new int[MAX];
7
8     // Constructor
9     stackArray_2511532010() {
10         top = -1;
11     }
12
13     boolean isEmpty() {
14         return (top == -1);
15     }
16
17     boolean push(int x) {
18         if (top >= (MAX - 1)) {
19             System.out.println("Stack Overflow");
20             return false;
21         } else {
22             a[++top] = x;
23             System.out.println(x + " dimasukkan dalam stack");
24             return true;
25         }
26     }
27
28     int pop_2511532010() {
29         if (top < 0) {
30             System.out.println("Stack Underflow");
31             return 0;
32         } else {
33             int x = a[top--];
34             return x;
35         }
36     }
37
38     int peek_2511532010() {
39         if (top < 0) {
40             System.out.println("Stack Underflow");
41             return 0;
42         } else {
43             int x = a[top];
44             return x;
45         }
46     }
47
48     void print() {
49         for (int i = top; i > -1; i--)
50         {
51             System.out.println (" " + a[i]);
52         }
53     }
54 }

```



```

1 package pekan3_2511532010;
2
3 public class stackArrayDriver_2511532010 {
4
5     public static void main(String[] args) {
6         stackArray_2511532010 s = new stackArray_2511532010();
7         s.push(10);
8         s.push(20);
9         s.push(30);
10
11         System.out.println(s.pop_2511532010() + " dikeluarkan dari stack");
12         System.out.println("Elemen teratas adalah : " + s.peek_2511532010());
13         System.out.println("Elemen pada stack :");
14         s.print();
15     }
16 }

```

1. Inisialisasi Objek Stack

Pada baris `stackArray_2511532010 s = new stackArray_2511532010()`, kita membuat tumpukan baru. Di balik layar, Java menyiapkan sebuah array berukuran 1000 slot dan mengatur variabel `top` ke angka -1. Ini adalah kondisi "nol" di mana tumpukan siap menerima data pertama.

2. Proses Pengisian (Push)

Kita memasukkan tiga nilai secara berturut-turut: 10, 20, dan 30.

- Saat 10 masuk, `top` naik ke 0.
- Saat 20 masuk, `top` naik ke 1.
- Saat 30 masuk, `top` naik ke 2.

Kini, angka 30 berada di posisi paling atas karena ia yang terakhir kita masukkan.

3. Eksekusi Operasi Pop

Baris `s.pop_2511532010()` dipanggil. Berdasarkan aturan LIFO, program akan melihat posisi `top` (indeks 2) dan mengambil nilainya, yaitu 30. Setelah itu, nilai `top` otomatis turun kembali ke 1. Angka 30 berhasil dikeluarkan dari struktur tumpukan kita.

4. Penggunaan Peek untuk Verifikasi

Setelah 30 dikeluarkan, kita memanggil `s.peek_2511532010()`. Hasilnya adalah 20. Ini membuktikan bahwa setelah elemen teratas diambil, elemen yang sebelumnya berada di bawahnya (20) kini naik menjadi elemen teratas yang baru.

5. Metode Cetak (Print)

Ketika kita memanggil `s.print()`, program melakukan iterasi mundur dari posisi top saat ini (indeks 1) menuju indeks 0. Hasil yang akan kita lihat di konsol adalah:
Plaintext : 20, 10

Urutan ini memastikan bahwa data yang ditampilkan selalu dari yang paling baru ke yang paling lama.

6. Keamanan Operasi Terurut

Melalui Driver ini, kita bisa melihat betapa konsistennya variabel top. Kita tidak bisa mengambil angka 10 selama angka 20 masih ada di atasnya. Inilah keindahan dari struktur data Stack; urutan data terjaga dengan sangat ketat.

7. Efisiensi Eksekusi

Karena kita menggunakan array statis, setiap operasi yang kita lakukan di dalam Driver ini (push, pop, peek) memiliki kompleksitas waktu $O(1)$. Artinya, program kita merespons secara instan tanpa peduli seberapa banyak data yang sudah ada di dalamnya.

8. Penanganan Pesan Konsol

Setiap kali kita melakukan push, metode di kelas utama mencetak konfirmasi "dimasukkan dalam stack". Ini sangat membantu kita untuk melacak pergerakan data di memori secara visual melalui terminal atau konsol.

9. Manajemen Status Underflow/Overflow

Meskipun di Driver ini kita hanya memasukkan 3 data, sistem kita sudah siap jika kita mencoba memasukkan lebih dari 1000 data (Overflow) atau mengambil data saat Stack sudah kosong (Underflow) berkat pengecekan kondisi yang sudah kita buat di kelas stackArray.

HASIL :

```
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah : 20
Elemen pada stack :
  20
  10
```

2.5 StackPostfix_2511532010

```

1 package pekan3_2511532010;
2
3 import java.util.Scanner;
4 import java.util.Stack;
5
6 public class StackPostfix_2511532010 {
7
8     public static int postfixEvaluate_2511532010(String expression) {
9         Stack<Integer> s = new Stack<>();
10        Scanner input = new Scanner(expression);
11
12        while (input.hasNext()) {
13            if (input.hasNextInt()) {
14                s.push(input.nextInt());
15            } else {
16                String operator = input.next();
17                int operand1 = s.pop();
18                int operand2 = s.pop();
19
20                if (operator.equals("+")) {
21                    s.push(operand2 + operand1);
22                } else if (operator.equals("-")) {
23                    s.push(operand2 - operand1);
24                } else if (operator.equals("*")) {
25                    s.push(operand2 * operand1);
26                } else if (operator.equals("/")) {
27                    s.push(operand2 / operand1);
28                }
29            }
30        }
31
32        input.close();
33        return s.pop();
34    }
35
36    public static void main(String[] args) {
37        System.out.println("hasil postfix= " + postfixEvaluate_2511532010("5 2 4 * + 7 -"));
38    }
39 }

```

1. Penggunaan Scanner untuk Parsing

Kita menggunakan Scanner `input = new Scanner(expression)` untuk memecah kalimat ekspresi menjadi bagian-bagian kecil (token). Dengan fungsi `input.hasNext()`, kita memastikan program terus berjalan selama masih ada angka atau operator yang perlu diproses dalam string tersebut.

2. Identifikasi Operand (Angka)

Melalui perintah `if (input.hasNextInt())`, kita mengecek apakah token yang sedang dibaca adalah angka. Jika benar, angka tersebut langsung kita masukkan ke

dalam tumpukan menggunakan `s.push()`. Angka-angka ini akan "menunggu" di tumpukan sampai ada operator yang memanggil mereka.

3. Pengambilan Operand saat Bertemu Operator

Ketika program menemukan operator (seperti `+`, `-`, `*`, `/`), kita melakukan `pop()` sebanyak dua kali. Hal yang perlu kita perhatikan adalah urutan pengambilan: `operand1` adalah angka yang paling atas, dan `operand2` adalah angka di bawahnya.

4. Pentingnya Urutan Pengurangan dan Pembagian

Pada operasi pengurangan dan pembagian, urutan sangat krusial. Karena Stack bersifat LIFO, angka yang masuk pertama berada di posisi `operand2`. Itulah sebabnya kita menulis `operand2 - operand1` dan `operand2 / operand1`. Jika terbalik, hasil perhitungan kita akan salah (misal: seharusnya $5-2$ malah menjadi $2-5$).

5. Logika Percabangan Operator

Kita menggunakan struktur `if-else if` untuk menentukan jenis perhitungan. Program akan mencocokkan string operator dengan simbol matematika yang sesuai. Setiap hasil perhitungan tersebut langsung kita masukkan kembali (`push`) ke dalam tumpukan sebagai hasil sementara.

6. Simulasi Eksekusi $5\ 2\ 4\ * + 7\ -$

Mari kita telusuri alur input di main:

- 5, 2, 4 masuk ke Stack.
- Bertemu `*` : 4 dan 2 di-pop, dikali ($2 \times 4 = 8$), 8 masuk Stack.

- Bertemu + : 8 dan 5 di-pop, ditambah ($5+8=13$), 13 masuk Stack.
- 7 masuk ke Stack.
- Bertemu -: 7 dan 13 di-pop, dikurang ($13-7=6$), 6 masuk Stack.

HASIL :

hasil postfix= 6

BAB III

KESIMPULAN

3.1 Ringkasan Hasil Praktikum

Melalui rangkaian praktikum ini, kita telah membuktikan bahwa Stack merupakan struktur data linier yang sangat krusial dalam manajemen memori dan pengaturan alur logika program melalui prinsip LIFO (Last In First Out). Berbagai latihan yang dilakukan menunjukkan bahwa operasi fundamental seperti push, pop, dan peek dapat diimplementasikan secara fleksibel, baik menggunakan library bawaan Java maupun melalui pendekatan manual dengan array statis dan ArrayList. Selain itu, kita memahami bahwa validasi kondisi isEmpty dan isFull menjadi aspek keamanan data yang wajib dipenuhi guna mencegah terjadinya galat fatal seperti overflow dan underflow.

Di samping aspek teknis dasar, praktikum ini juga memperlihatkan fleksibilitas Stack dalam memecahkan persoalan komputasi yang lebih kompleks. Penggunaan strategi tumpukan cadangan (backup stack) memberikan wawasan bagi kita tentang cara memproses data tanpa merusak integritas struktur aslinya. Sementara itu, penerapan Stack pada algoritma evaluasi Postfix menunjukkan efisiensi sistem dalam menangani hierarki perhitungan tanpa bantuan tanda kurung. Secara keseluruhan, penguasaan konsep ini menjadi fondasi yang kokoh bagi kita dalam memahami mekanisme internal komputer yang lebih dalam, seperti runtime stack, serta memperkuat kesiapan dalam menghadapi tantangan pemrograman pada level yang lebih lanjut.

DAFTAR PUSTAKA

- [1] Oracle, *The Java® Tutorials — Primitive Data Types*. [Online]. Available: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>
- [2] GeeksforGeeks, *Java Data Types – GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/data-types-in-java>

TUGAS

1. Class Website_2511532010

```
public class Website_2511532010 {  
    // Atribut dengan akhiran 4 digit NIM  
    private String judulWebsite_2010;  
    private String urlWebsite_2010;  
  
    // Constructor  
    public Website_2511532010(String judulWebsite_2010, String urlWebsite_2010) {  
        this.judulWebsite_2010 = judulWebsite_2010;  
        this.urlWebsite_2010 = urlWebsite_2010;  
    }  
  
    // Selektor (Getter)  
    public String getJudulWebsite_2010() {return judulWebsite_2010;}  
    public String getUrlWebsite_2010() {return urlWebsite_2010;}  
  
    // Mutator (Setter)  
    public void setJudulWebsite_2010(String judulWebsite_2010) {this.judulWebsite_2010 = judulWebsite_2010;}  
    public void setUrlWebsite_2010(String urlWebsite_2010) {this.urlWebsite_2010 = urlWebsite_2010;}
```

2. Class Browser_2511532010

```

public class Browser_2511532010 {
    public static void main(String[] args) {
        // Menggunakan Stack untuk menyimpan objek Website
        Stack<Website_2511532010> history = new Stack<>();
        Scanner input = new Scanner(System.in);
        int pilihan;

        do {
            System.out.println("\n=== Browser History NIM: 2511532010 ===");
            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");
            System.out.println("3. Lihat Halaman Aktif (Peek)");
            System.out.println("4. Cek Status History (Size/Empty)");
            System.out.println("5. Keluar");
            System.out.print("Pilihan: ");
            pilihan = input.nextInt();
            input.nextLine(); // Membersihkan buffer

            switch (pilihan) {
                case 1:
                    // Kunjungi Website (Push)
                    System.out.print("Masukkan Judul Website: ");
                    String judul = input.nextLine();
                    System.out.print("Masukkan URL Website: ");
                    String url = input.nextLine();

                    Website_2511532010 webBaru = new Website_2511532010(judul, url);
                    history.push(webBaru);
                    System.out.println("Website berhasil dikunjungi!");
                    break;

                case 2:
                    // Tombol Back (Pop)
                    if (!history.isEmpty()) {
                        Website_2511532010 hapus = history.pop();
                        System.out.println("Berhasil kembali dari: " + hapus.getJudulWebsite_2010());
                    } else {
                        System.out.println("History kosong, tidak bisa kembali!");
                    }
                    break;

                case 3:
                    // Lihat Halaman Saat Ini (Peek)
                    if (!history.isEmpty()) {
                        Website_2511532010 aktif = history.peek();
                        System.out.println("\n--- Halaman Aktif Saat Ini ---");
                        System.out.println("Judul: " + aktif.getJudulWebsite_2010());
                        System.out.println("URL : " + aktif.getUrlWebsite_2010());
                    } else {
                        System.out.println("Tidak ada halaman yang sedang dibuka.");
                    }
                    break;

                case 4:
                    // Cek Status History
                    if (history.isEmpty()) {
                        System.out.println("Status: Riwayat penjelajahan kosong.");
                    } else {
                        System.out.println("Status: Terdapat " + history.size() + " halaman dalam riwayat.");
                    }
                    break;

                case 5:
                    System.out.println("Menutup Browser...");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan != 5);

        input.close();
    }
}

```

HASIL :

```

=== Browser History NIM: 2511532010 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History (Size/Empty)
5. Keluar
Pilihan: 1
Masukkan Judul Website: SSO UNAND
Masukkan URL Website: https://sso.unand.ac.id/login
Website berhasil dikunjungi!

=== Browser History NIM: 2511532010 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History (Size/Empty)
5. Keluar
Pilihan: 1
Masukkan Judul Website: SHOPEE INDONESIA
Masukkan URL Website: https://shopee.co.id
Website berhasil dikunjungi!

=== Browser History NIM: 2511532010 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History (Size/Empty)
5. Keluar
Pilihan: 2
Berhasil kembali dari: SHOPEE INDONESIA

=== Browser History NIM: 2511532010 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History (Size/Empty)
5. Keluar
Pilihan: 3

--- Halaman Aktif Saat Ini ---
Judul: SSO UNAND
URL : https://sso.unand.ac.id/login

=== Browser History NIM: 2511532010 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History (Size/Empty)
5. Keluar
Pilihan: 4
Status: Terdapat 1 halaman dalam riwayat.

=== Browser History NIM: 2511532010 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History (Size/Empty)
5. Keluar
Pilihan:

```